

01. 统计学习方法

本篇笔记由【三黄工作室】微信公众号提供。欢迎关注。



统计学习由监督学习、非监督学习、半监督学习和强化学习等组成。

1. 第一章 统计学习方法概论

从给定的、有限的、用于学习的训练数据集合出发，**假设数据是独立同分布产生的**；并且假设要学习的模型属于某个函数的集合，也就是假设空间；应用某个评价准则，从假设空间中选取一个最优的模型，使它对已知训练数据及未知测试数据在给定的评价准则下有最优的预测；最优模型的选取由算法实现。这样，统计学习方法包括

- 模型的假设空间（模型结构）
- 模型选择的准则（损失函数或评价指标）
- 模型学习的算法（模型更新算法）



独立同分布要拆开理解，“独立”表示两条样本间不存在依赖关系，是独立发生的；“同分布”表示样本的产生都是来自于同一个概率分布中。

独立同分布只是我们假定的理想情况，现实中我们的训练集都是有偏的，比如文本分类的训练集可能体育标签的数据更多、大模型训练数据从网页爬取的数据更多。

1.1 监督学习

监督学习主要包含以下三类典型问题：

1. **分类问题**：输出是离散值。

2. **回归问题**：输出是连续值。常用模型是线性回归 LR、决策树回归 CART、支持向量回归 SVR。
3. **标注问题**：输入是观测序列，输出是标记序列。常见的任务是 NER，常用模型是隐马尔可夫模型 HMM、条件随机场 CRF。

1.1.1 联合概率分布

监督学习的基本假设：输入和输出遵循联合概率分布 $P(X, Y)$ ，也就是说假定任务数据一定满足一个规律，并且这些数据就是依照联合概率分布 $P(X, Y)$ 独立同分布产生的。这是基本假设，也是当数据量趋于无穷时，期望能够达到的水平。

联合概率分布 $P(X, Y)$ 指的是 X 和 Y 同时出现的概率。条件概率分布 $P(Y|X)$ 指的是已知 X 出现的情况下， Y 出现的概率。

虽然监督学习都基于这样的基本假设，但在实际算法模型中，不同模型建模的目标却是不同的。

1.2 生成式与判别式

监督学习的模型可以是概率模型或者非概率模型。

- 如果是判别式模型，建模方法由条件概率分布 $P(Y|X)$ 或决策函数 $y = f(x)$ 表示。
- 如果是生成式模型，建模的是联合概率分布 $P(X, Y)$ 。

区别就在于建模的概率不一样，一个是条件概率，另外一个联合概率。

在贝叶斯定理下，虽然条件概率分布和联合概率分布有一定的关联。但是判别式模型所建模的条件概率是可以直接用数学公式表示出来的，但是生成式模型所建模的联合概率分布必须通过先验概率和似然来间接表达出来。

注：贝叶斯定理下，联合概率分布 $P(X, Y) = P(X | Y)P(Y)$ ，后验概率

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)}。$$

💡 大模型也属于生成式模型，本质上是对观测序列 X 的联合分布 $P(X)$ 建模，它和狭义的生成式模型的区别还是比较大的。

1.3 问题的形式化

首先给定一个训练数据集

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

其中 (x_i, y_i) , $i = 1, 2, \dots, N$, 称为样本或样本点。 $x_i \in \mathcal{X} \subseteq \mathbf{R}^n$ 是输入样本, $y_i \in \mathcal{Y}$ 是标签。假设训练数据与测试数据是依联合概率分布 $P(X, Y)$ 独立同分布产生的。

在学习过程中, 学习系统利用给定的训练数据集, 通过学习 (或训练) 得到一个模型, 表示为条件概率分布 $\hat{P}(Y|X)$ 或决策函数 $Y = \hat{f}(X)$ 。条件概率分布 $\hat{P}(Y|X)$ 或决策函数 $Y = \hat{f}(X)$ 描述输入与输出随机变量之间的映射关系。

在预测过程中, 预测系统对于给定的测试样本集中的输入 x_{N+1} , 由模型

$y_{N+1} = \arg \max_{y_{N+1}} \hat{P}(y_{N+1}|x_{N+1})$ 或 $y_{N+1} = \hat{f}(x_{N+1})$ 给出相应的输出 y_{n+1} 。

1.3.1 损失函数

统计学习常用的损失函数有以下几种:

(1) 0-1 损失函数 (0-1 loss function)

$$L(Y, f(X)) = \begin{cases} 1, & Y \neq f(X) \\ 0, & Y = f(X) \end{cases}$$

(2) 平方损失函数 (quadratic loss function)

$$L(Y, f(X)) = (Y - f(X))^2$$

(3) 绝对损失函数 (absolute loss function)

$$L(Y, f(X)) = |Y - f(X)|$$

(4) 对数损失函数 (logarithmic loss function) 或对数似然损失函数 (log-likelihood loss function)

$$L(Y, P(Y|X)) = -\log P(Y|X)$$

1.3.2 期望风险

因为模型的输入、输出是随机变量, 而且遵循联合分布, 所以**损失函数的期望是在样本空间上求积分**

$$R_{\text{exp}}(f) = E_P[L(Y, f(X))] = \int_{\mathcal{X} \times \mathcal{Y}} L(y, f(x)) P(x, y) dx dy$$

在概率论中, “期望”表示一组随机变量在概率分布下的加权平均值, 也是理想值, 只能通过在样本量趋向于无穷时来逼近, 而不能达到。

1.3.3 经验风险

正因为期望风险无法直接计算，所以在训练集上计算经验风险，来估计期望风险。

模型 $f(X)$ 关于训练数据集的平均损失称为**经验风险**或**经验损失**，记作 R_{emp} ：

$$R_{\text{emp}}(f) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i))$$

期望风险 $R_{\text{exp}}(f)$ 是模型关于联合分布的期望损失，经验风险 $R_{\text{emp}}(f)$ 是模型关于训练样本集的平均损失。

经验风险是我们实际使用的 loss，期望风险属于理论值。

1.3.4 经验风险最小化与结构风险最小化

根据大数定律，当样本容量 N 趋于无穷时，经验风险趋向于期望风险，但是由于样本数据有限，甚至很小，所以用经验风险估计期望风险常常并不理想，要对经验风险进行一定的矫正，这就关系到监督学习的两个基本策略：经验风险最小化和结构风险最小化。

经验风险最小化（empirical risk minimization，**ERM**）的策略按照经验风险最小化求最优模型就是求解最优化问题：

$$\min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i))$$

当样本容量很小时，经验风险最小化学习的效果就未必很好，会产生过拟合。

结构风险最小化（structural risk minimization，**SRM**）是为了防止过拟合而提出来的策略，结构风险最小化等价于正则化（regularization）。结构风险在经验风险上加上表示模型复杂度的正则化项或罚项，定义是：

$$R_{\text{erm}}(f) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i)) + \lambda J(f)$$

结构化风险的正则项一般采用模型参数的 L1 距离或者 L2 距离。

极大似然估计（Maximum Likelihood Estimation，MLE）就是**经验风险最小化**的一个例子，当模型是条件概率分布，损失函数是对数损失函数时，经验风险最小化就等价于极大似然估计。

证明：

1) ERM

假设模型是条件概率分布，损失函数使用对数损失：

$$L(y, P_{\theta}(y | x)) = -\log P_{\theta}(y | x)$$

经验风险定义为在训练集上的平均损失：

$$R_{\text{emp}}(\theta) = \frac{1}{n} \sum_{i=1}^n L(y_i, P_{\theta}(y_i | x_i)) = \frac{1}{n} \sum_{i=1}^n [-\log P_{\theta}(y_i | x_i)]$$

最小化经验风险表示求模型参数以能够得到最小的经验风险，即：

$$\hat{\theta}_{\text{ERM}} = \arg \min_{\theta} R_{\text{emp}}(\theta) = \arg \min_{\theta} \left(-\frac{1}{n} \sum_{i=1}^n \log P_{\theta}(y_i | x_i) \right)$$

忽略掉常数，等价于

$$\hat{\theta}_{\text{ERM}} = \arg \max_{\theta} \left(\sum_{i=1}^n \log P_{\theta}(y_i | x_i) \right)$$

2) MLE

在统计学中，给定模型 $P_{\theta}(y | x)$ ，似然函数定义为所有观测样本的联合条件概率（假设样本独立同分布）：

$$\mathcal{L}(\theta) = \prod_{i=1}^n P_{\theta}(y_i | x_i)$$

取对数得对数似然：

$$\log \mathcal{L}(\theta) = \sum_{i=1}^n \log P_{\theta}(y_i | x_i)$$

极大似然估计就是：

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \sum_{i=1}^n \log P_{\theta}(y_i | x_i)$$

因此等价。当模型是条件概率分布、损失函数是对数损失时，经验风险最小化等价于极大似然估计。

既然有了经验风险最小化，为什么还要有最大似然估计？

首先，“似然”给了 loss 一个“合法性来源”，有了似然，可以说这个 loss 是从一个概率模型 + 数据生成假设推导出来的。如，MSE 是假设数据服从高斯分布，分类的交叉熵损失服从伯努利分布。

其次，最大似然估计+先验分布=最大后验，这个是生成式模型的核心思想。

最后，两者本质都在“让模型更好地拟合观测数据的概率分布”，只是从不同角度表述：

- MLE：概率建模视角（生成/判别式概率模型）。
- ERM：机器学习优化视角（最小化预期损失）。

1.4 正则化

正则化的理论思想是选择参数复杂度低的模型。

正则化是结构风险最小化策略的实现，是在经验风险上加一个正则化项（regularizer）或罚项（penalty term）。正则化项一般是模型复杂度的单调递增函数，模型越复杂，正则化值就越大。比如，正则化项可以是模型参数向量的范数。

正则化一般具有如下形式：

$$\min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i)) + \lambda J(f)$$

正则化项可以取不同的形式。例如，回归问题中，损失函数是平方损失，正则化项可以是参数向量的 L_2 范数：

$$L(w) = \frac{1}{N} \sum_{i=1}^N (f(x_i; w) - y_i)^2 + \frac{\lambda}{2} \|w\|^2$$

正则化项也可以是参数向量的 L_1 范数：

$$L(w) = \frac{1}{N} \sum_{i=1}^N (f(x_i; w) - y_i)^2 + \lambda \|w\|_1$$

正则化的作用是选择经验风险与模型复杂度同时较小的模型。

2. 第二章 感知机

定性地说：感知机是二分类线性分类模型，它是判别式模型，是神经网络和支持向量机的基础。

感知机是 SVM、神经网络的基础。

感知机和线性回归有什么区别？

它们的核心都是：

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

线性回归的输出就是浮点数 $f(x)$ ，而感知机的输出是 $\text{sign}(f(x))$ ，即把浮点数转化为离散值。

2.1 感知机模型

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

其中， $\text{sign}()$ 是符号函数

$$\text{sign}(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

感知机在 $x=0$ 时不可导，因此无法用于基于梯度的优化（如反向传播）。

正因 `sign` 不可导，神经网络不再使用它作为激活函数，而改用可导的近似（如 tanh、ReLU、sigmoid）。

2.2 感知机学习算法

2.2.1 数据集的线性可分性

给定的数据集中，假设输入 x 是 n 维的，如果一定有一个 n 维的超平面可以讲这些样本划分到超平面的两侧，就说这个数据集是线性可分的。

如果数据集不是线性可分的，那么感知机永远不会停止更新，无法收敛。

此时应该引入非线性激活函数，或者换其他模型。

2.2.2 损失函数

由于原模型在 $x=0$ 不可导，采用误分类点到平面的 L2 距离来作为感知机的损失函数。

首先，写出输入空间 $\mathbb{R}^n$ 中任一点 x_0 到超平面 S 的 L2 距离：

$$\frac{1}{\|w\|} |w \cdot x_0 + b|$$

由于 `sign` 的特性，误分类点 x_i 到超平面 S 的距离可以拆开绝对值，是

$$-\frac{1}{\|w\|} y_i (w \cdot x_i + b)$$

这样，假设超平面 S 的误分类点集合为 M ，那么所有误分类点到超平面 S 的总距离为

$$-\frac{1}{\|w\|} \sum_{x_i \in M} y_i (w \cdot x_i + b)$$

不考虑 $\frac{1}{\|w\|}$ ，就得到感知机学习的损失函数，定义为

$$L(w, b) = - \sum_{x_i \in M} y_i (w \cdot x_i + b)$$

其中 M 为误分类点的集合。这个损失函数就是感知机学习的经验风险函数。

如果考虑 $\frac{1}{\|w\|}$ ，那么最优化的目标就是一个离各个的的总距离都最小的平面，如果不考虑，最优化的目标就是只要能找到划分两类样本的平面即可。

2.2.3 优化算法

假设误分类点集合 M 是固定的，那么损失函数 $L(w, b)$ 的梯度为

$$\nabla_w L(w, b) = - \sum_{x_i \in M} y_i x_i$$

$$\nabla_b L(w, b) = - \sum_{x_i \in M} y_i$$

随机选取一个误分类点 (x_i, y_i) ，对 w, b 进行更新：

$$w \leftarrow w + \eta y_i x_i$$

$$b \leftarrow b + \eta y_i$$

式中 η 为学习率。

2.3 代码实现

[https://github.com/Unagi-cq/th-nlp-](https://github.com/Unagi-cq/th-nlp-learning/tree/main/1.%20%E7%BB%9F%E8%AE%A1%E5%AD%A6%E4%B9%A0%E6%96%B9%E6%B3%95/codes)

[learning/tree/main/1.%20%E7%BB%9F%E8%AE%A1%E5%AD%A6%E4%B9%A0%E6%96%B9%E6%B3%95/codes](https://github.com/Unagi-cq/th-nlp-learning/tree/main/1.%20%E7%BB%9F%E8%AE%A1%E5%AD%A6%E4%B9%A0%E6%96%B9%E6%B3%95/codes)

2.4 思考

巧思：假设初始 w 和 b 为 0，且迭代时重复循环遍历训练集。当学习率 or 输入 x or 输出 y 乘以一个因子 c ，模型收敛迭代的次数、最终的参数 w 和 b 是否会有变化，会有怎样的变化？

答：

此时 $w_{final} = \sum \eta y_i x_i$ 、 $b_{final} = \sum \eta y_i$

- 如果是学习率或输出 y 乘以因子 c ，因为模型只关心正负，迭代次数不变，但是 w 和 b 也会同乘以 c 。
- 如果是输入 x 乘以因子 c ，迭代次数会变，但是 w 和 b 也会变化，这是因为偏置 b 的尺度没有放缩。

可以调上面代码里面的参数看看。

3. 第三章 k 近邻法

定性地说：kNN 是分类与回归方法，它是判别式模型，不具有学习过程，而是对数据集预先生成一种划分规则，以便于预测新样本。

在嵌入模型与 RAG 火起来之后，通过 ES 来召回 chunk 是生产实践的首选，ES 里面用的 ANN 是近似最近邻，是对大规模数据做 knn 的近似拟合，常用算法有 HNSW、FAISS。

3.1 kNN 算法

给定训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ ，输入 x ，求 x 对应的类别 y 。

kNN 算法的步骤如下：

(1) 根据给定的距离度量，在训练集 T 中找出与 x 最邻近的 k 个点，涵盖这 k 个点的 x 的邻域记作 $N_k(x)$ ；

(2) 在 $N_k(x)$ 中根据分类决策规则（如多数表决）决定 x 的类别 y ：

$$y = \arg \max_{c_j} \sum_{x_i \in N_k(x)} I(y_i = c_j), \quad i = 1, 2, \dots, N; \quad j = 1, 2, \dots, K$$

式中， I 为指示函数，即当 $y_i = c_j$ 时 I 为 1，否则 I 为 0。

kNN 法的特殊情况是 $k=1$ 的情形，称为最近邻算法。

因此，kNN 算法最重要的三要素就是距离度量、 k 、分类决策规则。

3.1.1 距离度量

kNN 的性能高度依赖于所选的距离度量方式。常用的距离计算方法有 L1 距离、L2 距离、 L_p 距离和余弦相似度。

3.1.2 k 值的选择

k 值过小时，模型容易过拟合，所有预测容易太像训练集上的单点的分布。

k 值过大时，模型欠拟合，所有预测都变成多分类。

3.1.3 分类决策规则

对于分类任务，kNN 的预测基于 k 个最近邻的标签，常见决策规则有：

- 多数投票 (Majority Voting)
选择 k 个邻居中出现次数最多的类别作为预测结果。这是最常用的方式。
- 加权投票 (Weighted Voting)
距离越近的邻居权重越大，例如权重设为 $w_i = 1/d_i$ 或 $w_i = 1/d_i^2$ ，然后按加权票数决定类别。

回归任务中，通常用 k 个邻居目标值的平均值或加权平均值作为预测。

3.2 kNN 的实现：kd 树

当数据集的数据很多时，不可能去循环遍历来判断一个输入点和数据集上每个点的距离，性能无法接受，因此需要一些特殊的数据结构来帮助更高效的查找 k 近邻。这就是 kd 树 (k dimension tree)。

kd 树 (k-dimensional tree) 是一种用于加速 kNN 搜索的空间划分数据结构，特别适用于低维（通常 $d < 20$ ）的欧氏空间。

 当维度高时，kd 树难以剪枝，导致性能退化。

3.2.1 构建

kd 树是一种二叉空间分割树，通过递归地选择一个维度，并在该维度上选取中位数作为切分点，将空间划分为两个子空间。

1. 选择切分维度

通常采用轮换方式（如第 0 层按第 0 维，第 1 层按第 1 维，…，第 d 层回到第 0 维），也可以选择方差最大的维度以获得更平衡的划分。

2. 选择切分点

在选定维度上，取当前点集的**中位数**作为切分值，使得左右子树尽可能平衡（保证树高度为 $O(\log n)$ ）。

3. 递归构建左右子树

将小于切分值的点放入左子树，大于的放入右子树（等于可任意分配，但需保持一致性）。

4. 终止条件

当前节点包含的点数为 0 或 1 时停止递归。

3.2.2 查找

kd 树的查找分为最近邻和 k 近邻。

（最近邻，k=1）

1. **向下搜索**：从根节点开始，按照查询点在各维度上的值，递归进入左或右子树，直到叶子节点。

2. **向上回溯**：

- 计算当前节点到查询点的距离，更新“当前最近点”。
- 检查另一侧子树是否可能包含更近的点：若查询点到当前切分超平面的距离 **小于** 当前最近距离，则需搜索另一侧子树。**设当前节点切分维度为 `axis`，切分值为 `split_val`，查询点为 `q`，当前最近距离为 `best_dist`。若 `|q[axis] - split_val| < best_dist`，则超平面另一侧可能存在更近点，需递归搜索。**

3. **重复回溯**，直到根节点。



剪枝思想的原理：

上面介绍的剪枝思想只适用于 L1 距离、L2 距离和 Lp 距离，这是由计算公式导致的。如果某一个维度的距离为 Z，那么这三个距离公式最终一定是大于等于 Z。

也就是说，`|q[axis] - split_val|` 代表查询点到分割面的距离，一旦它比当前最近距离为 `best_dist` 还大，那么 q 点到分割面右侧的距离只会更大，就没必要算了，因此可以剪枝。

扩展到 k 近邻

- 维护一个大小为 k 的**最大堆**（或排序列表），保存当前最近的 k 个点及其距离。
- 每次找到更近的点，就更新堆。
- 回溯时，用堆顶（最大距离）代替 `best_dist` 作为剪枝阈值。

3.3 代码实现

<https://github.com/Unagi-cq/th-nlp-learning/tree/main/1.%20%E7%BB%9F%E8%AE%A1%E5%AD%A6%E4%B9%A0%E6%96%B9%E6%B3%95/codes>

3.4 思考

问题：kd 树的性能取决于什么？

答：当维度变大时，kd 树的剪枝效率会变低，直至退化成暴力搜索。维度 d 的临界值在 $\log_2^n$ ，n 是样本数量，当 d 超过临界值后，kd 树的效率会急剧降低。即理想情况是每个维度最多使用一次。

问题：kd 树的输入如果同乘以一个因子，会怎么样？

答：kd 树的性能是否受输入放缩的影响，要看距离公式和剪枝条件。虽然在构造 kd 树时本质上是在用中位数划分空间，但是在搜索时如果距离计算和剪枝条件的值不是一起放缩的话，就会有影响。

问题：距离度量函数和剪枝条件有什么关系？

答：如果距离函数是 L1、L2、切比雪夫距离，那么标注 kd 树剪枝条件都成立，因为本质是利用距离函数在某一维度上的局部下界来排除整个子空间。但如果是余弦相似度，就不适用了。

4. 第四章 朴素贝叶斯法

朴素贝叶斯法是十分标准的生成式模型。

朴素贝叶斯 = 概率论恒等式 + 贝叶斯估计 + 条件独立性假设。概率模型理论是后续学习大模型与蒸馏等知识的基础。

概率论恒等式：

$$P(X, Y) = P(Y) \cdot P(X|Y)$$

利用贝叶斯定理，可以推导出后验（即条件概率）：

$$P(Y|X) = \frac{P(Y)P(X|Y)}{P(X)} \propto P(Y)P(X|Y)$$

因此，生成式模型通过建模先验和似然来间接得到 $P(Y|X)$ 。但是由于似然的计算是指数级别的，几乎不可计算，所以再加上条件独立性假设，似然就可以通过各个参数的乘积来求得。

概念澄清

在分类任务上，先验概率 $P(Y)$ 指类别标签的概率，证据 $P(X)$ 指的是输入特征的概率，这两个都是边缘概率，也就是无条件概率，可以直接从数据集中统计（也叫做极大似然估计）出来。后验概率 $P(Y|X)$ 和似然 $P(X|Y)$ 形式上都属于条件概率。

4.1 朴素贝叶斯算法

朴素贝叶斯法分类时，对给定的输入 x ，通过学习到的模型计算后验概率分布 $P(Y = c_k | X = x)$ ，将后验概率最大的类作为 x 的类输出。后验概率计算根据贝叶斯定理进行：

$$P(Y = c_k | X = x) = \frac{P(X = x | Y = c_k)P(Y = c_k)}{\sum_k P(X = x | Y = c_k)P(Y = c_k)}$$

又由于似然可以用条件独立性假设作连乘表示，且分母可以忽略，最终朴素贝叶斯分类器可表示为

$$y = \arg \max_{c_k} P(Y = c_k) \prod_j P(X^{(j)} = x^{(j)} | Y = c_k)$$

4.2 朴素贝叶斯法的参数估计

我们不知道也不可能知道 $P(Y = c_k)$ 和 $P(X^{(j)} = x^{(j)} | Y = c_k)$ 的真实分布，但是可以用数据集中的数据分布来去做估计。

4.2.1 极大似然估计

因此，先验概率 $P(Y = c_k)$ 的极大似然估计就是数据集中类别 c_k 的分布概率。而每个特征的似然 $P(X^{(j)} = x^{(j)} | Y = c_k)$ 就是数据集中类别 c_k 下特征 j 的真实占比。

也就是用频率估计概率。

4.2.2 贝叶斯估计

当使用极大似然估计可能会出现所要估计的概率值为 0 的情况。这时会影响到后验概率的计算结果，使分类产生偏差。解决这一问题的方法是采用贝叶斯估计。具体地，条件概率的贝叶斯估计是

$$P_\lambda(X^{(j)} = a_{jl} | Y = c_k) = \frac{\sum_{i=1}^N I(x_i^{(j)} = a_{jl}, y_i = c_k) + \lambda}{\sum_{i=1}^N I(y_i = c_k) + S_j \lambda}$$

等价于在随机变量各个取值的频数上赋予一个正数 $\lambda > 0$ 。当 $\lambda = 0$ 时就是极大似然估计。常取 $\lambda = 1$ ，这时称为拉普拉斯平滑（Laplace smoothing）。

4.3 代码实现

<https://github.com/Unagi-cq/th-nlp-learning/tree/main/1.%20%E7%BB%9F%E8%AE%A1%E5%AD%A6%E4%B9%A0%E6%96%B9%E6%B3%95/codes>

5. 第五章 决策树

决策树是判别式模型，既可以分类，也可以做回归任务。

决策树可以理解为一堆 if-then 规则的集合体，这体现为“决策边界”，也可以理解为条件概率分布。

在生产实践中，由于有些场景对算法模型的响应速度有极高的要求，有时候会使用 XGBoost 算法，而决策树（尤其是 CART）正是 XGBoost 的基础。

决策树、KD 树、KNN

决策树是规则树模型，它划分的是“特征-标签”空间。

而 KD 树只是一种数据结构，它只划分特征空间，有点像无监督的感觉。KD 树只是为了加速 KNN 来搜索近邻的工具，KNN 理论上是要遍历全部训练集来找 K 近邻的。

因此，决策树和 KD 树、KNN 没啥关系。

假设有一批训练集，输入有 10 个特征，输出是一个类别。决策树首先要选举出一个和类别最相关的特征，然后以它作标准，划分出若干子树，然后递归的分下去。但是这样划分的树太精细（过拟合），所以最后还要剪枝。

决策树的学习包括 3 个步骤：特征选择、决策树的生成和剪枝。

比较经典的三个决策树模型是 ID3、C4.5 和 CART，它们是层层递进的关系。

5.1 熵

有关于特征的选择，涉及到熵，因此熵是决策树的前置知识。

5.1.1 信息熵

在信息论中，熵表示一个随机变量的不确定性程度，也说是混乱程度。

离散变量和连续变量都有熵，离散变量的熵是离散求和形式，连续变量的熵是微分形式。后面主要讲离散变量的熵。

假设一个离散变量的概率分布为 $p(X = x_i) = p_i, i = 1, \dots, n$ ，那么它的信息熵定义为（注意 p_i 之和为 1）

$$H(X) = - \sum p_i * \log(p_i)$$

有关于熵，有两个重要的理解：

1. **信息熵的取值是否有最大值？** 有，当所有可能的取值的概率都是 $\frac{1}{n}$ 时，取最大值 $\log(n)$ ，所以理论上信息熵是个无上限的值。
2. **当新增一个取值时，信息熵是否一定增加？** 是，只要新增的取值具有正概率（哪怕非常小），信息熵一定会增加，但增幅很小，且随着新取值概率趋于 0，信息熵的增量也趋于 0。

证明

设原随机变量 X 有 n 个取值，概率分布为 $p_1, p_2, \dots, p_n$ ，熵为：

$$H(X) = - \sum_{i=1}^n p_i \log p_i.$$

现在引入一个新取值 x_{n+1} ，其概率为 $\varepsilon > 0$ （很小），为了保持概率和为 1，需要从原分布中“挪出” ε 的概率质量。

即新变量 X' 的分布为：

$$P(X' = x_i) = (1 - \varepsilon)p_i \quad (i = 1, \dots, n), \quad P(X' = x_{n+1}) = \varepsilon.$$

这相当于对原分布做了缩放，并新增一个低概率事件。此时新熵为：

$$H(X') = - \sum_{i=1}^n (1 - \varepsilon)p_i \log[(1 - \varepsilon)p_i] - \varepsilon \log \varepsilon.$$

展开后：

$$\begin{aligned} H(X') &= -(1 - \varepsilon) \log(1 - \varepsilon) - (1 - \varepsilon) \sum_{i=1}^n p_i \log p_i - \varepsilon \log \varepsilon. \\ &= (1 - \varepsilon)H(X) - (1 - \varepsilon) \log(1 - \varepsilon) - \varepsilon \log \varepsilon. \end{aligned}$$

由于：

- $-\log(1 - \varepsilon) \approx \varepsilon$ （当 ε 很小时），
- $-\varepsilon \log \varepsilon > 0$ 且主导项（因为 $\varepsilon \log \varepsilon \rightarrow 0^-$ ）

所以 $H(X') > H(X)$ 。

5.1.2 信息熵、交叉熵与 KL 散度

在 NLP 领域，熵的概念通常会涉及到信息熵、交叉熵、KL 散度，常见的分类任务损失函数和蒸馏任务的损失函数与熵都有密切的关系。

1. 信息熵

信息熵表示一个变量的**不确定性**或**信息量的期望**。熵越大，分布越“混乱”；熵越小，分布越集中。

2. 交叉熵

给定两个概率分布 P （真实分布）和 Q （模型预测分布），交叉熵衡量分布 Q 来编码来自 P 的样本所需的平均比特数。

$$H(P, Q) = - \sum_x P(x) \log Q(x)$$

在监督学习中， P 通常是 one-hot 标签（真实类别）， Q 是模型输出的概率。

3. KL 散度

KL 散度衡量两个分布之间的差异：

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = H(P, Q) - H(P)$$

KL 散度是非对称的； $D_{\text{KL}} \geq 0$ ；当且仅当 $P = Q$ 时为 0。



提示

在文本分类任务中，无论是二分类、多分类还是多标签分类，我们使用的都是交叉熵损失函数。如果我们使用 KL 散度，由于类别标签是 0-1 的独热编码，代入计算 $H(P) = 0$ ，因此 KL 散度就等价于交叉熵。

或者从理论的角度解释，因为类别标签的分布是确定的，其熵 $H(P) = 0$ ，因此 KL 散度就等价于交叉熵。

现在我们用大模型去蒸馏小模型时，是让其尽可能地拟合大模型给出的分布，并且大模型的输出也是概率形式，并不是确定性的，因此用的是蒸馏损失，即 KL 散度。

5.1.3 条件熵

条件熵（Conditional Entropy）是信息论中用于衡量在已知一个随机变量的条件下，另一个随机变量的**不确定性的量**。

设 X 和 Y 是两个离散随机变量，**条件熵** $H(Y|X)$ 定义为：

$$\begin{aligned} H(Y|X) &= \sum_{x \in \mathcal{X}} P(x) H(Y|X = x) \\ &= - \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} P(x, y) \log P(y|x) \end{aligned}$$

信息增益表示得知特征 A 的信息而使得类 Y 的信息不确定性减少的程度。特征 A 对训练数据集 D 的信息增益 $g(D, A)$ ，定义为集合 D 的经验熵 $H(D)$ 与特征 A 给定条件下 D 的经验条件熵 $H(D | A)$ 之差，即

$$g(D, A) = H(D) - H(D | A)$$

熵 $H(Y)$ 与条件熵 $H(Y | X)$ 之差称为**互信息**（mutual information）。决策树学习中的信息增益等价于训练数据集中类与特征的互信息。

信息增益比是信息增益与训练集的经验熵之比：

$$g(D, A) = \frac{g(D, A)}{H(D)}$$

5.2 ID3 与 C4.5

ID3（Iterative Dichotomiser 3）算法由 Ross Quinlan 提出，是一种**自顶向下、贪心**的决策树构建方法，其核心是使用**信息增益**（Information Gain）作为特征选择标准。

ID3 构建决策树的详细步骤

1. **输入**：一个训练数据集 D ，包含特征集合 $A = \{A_1, A_2, \dots, A_n\}$ 和类别标签 Y 。
2. **创建根节点**：将整个数据集 D 作为当前节点的数据。
3. **检查停止条件**（若满足任一，则生成叶节点）：
 - **情况 1**：如果 D 中所有样本属于同一类别 c ，则将当前节点标记为**叶节点**，类别为 c 。
 - **情况 2**：如果特征集合 A 为空（即无可分特征），或所有样本在 A 上取值相同，则将当前节点标记为**叶节点**，类别为 D 中**出现频率最高的类别**。
4. **选择最优划分特征**（核心步骤）：
 - 对每个候选特征 $A_i \in A$ ，计算其**信息增益** $g(D, A_i) = H(D) - H(D | A_i)$ 。
 - 选择**信息增益最大**的特征 A^* 作为当前节点的划分标准。
5. **生成子节点**：
 - 以 A^* 的每一个可能取值 v_j 为分支，将数据集 D 划分为若干子集 D_{v_j} （即 $A^* = v_j$ 的样本）。
 - 对每个非空子集 D_{v_j} ，递归地以 D_{v_j} 为新的数据集，并从特征集合 A 中移除 A^* （因为 ID3 不支持特征重复使用），重复步骤 2-5。
6. **输出**：最终得到一棵完整的决策树。

- **仅支持离散特征**：ID3 无法直接处理连续型变量，需先进行离散化。
- **偏向高基数特征**：信息增益倾向于选择取值多的特征（如“用户 ID”），这可能导致过拟合。

C4.5 的流程和 ID3 相似，C4.5 是对 ID3 的系统性改进：用信息增益率代替信息增益，支持连续特征和缺失值，并加入剪枝机制，从而缓解 ID3 偏好多值特征和过拟合的问题。



ID3 只支持离散型输入，如遇到连续特征，需要做分桶。C4.5 支持离散输入和连续输入，遇离散特征会分裂为多叉，遇连续特征会分裂为二叉。

5.3 CART

ID3、C4.5 算法都属于简单决策树算法，并且无论是在实际生产中还是标准库实现中，基本上决策树底层实现都用的是 CART。提及剪枝，我们通常也只讨论 CART 的剪枝。

因为 CART 的二叉结构和成本复杂度剪枝更简单、稳定，且更容易扩展到随机森林和梯度提升，因此在工程中取代了 C4.5。

例如，sklearn 中的 `DecisionTreeClassifier` 和 `DecisionTreeRegressor` 基于 CART 算法，采用二叉划分，分类任务支持 Gini 或 Entropy 作为指标，并使用成本复杂度剪枝（`ccp_alpha`）。

算法 / 特性	ID3	C4.5	CART	sklearn
二叉树	✗	✗	✓	✓
多叉树	✓	✓	✗	✗
Gini	✗	✗	✓	✓
Entropy	✓	✓	✗	✓（但仍是 CART）
回归树	✗	✗	✓	✓
成本复杂度剪枝	✗	✗	✓	✓

CART 树既可以做分类，也可以做回归，并且它还接受离散特征和连续特征混合输入。

5.3.1 回归树的生成

对回归树用平方误差最小化准则

回归树生成算法：在训练数据集所在的输入空间中，递归地将每个区域划分为两个子区域并决定每个子区域上的输出值，构建二叉决策树：

(1) 选择最优切分变量 j 与切分点 s ，求解

$$\min_{j,s} \left[\min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2 \right]$$

遍历变量 j ，对固定的切分变量 j 扫描切分点 s ，选择上式达到最小值的对 (j, s) 。

(2) 用选定的对 (j, s) 划分区域并决定相应的输出值：

$$R_1(j, s) = \{x \mid x^{(j)} \leq s\}, \quad R_2(j, s) = \{x \mid x^{(j)} > s\}$$

$$\hat{c}_m = \frac{1}{N_m} \sum_{x_i \in R_m(j,s)} y_i, \quad x \in R_m, \quad m = 1, 2$$

(3) 继续对两个子区域调用步骤 (1)，(2)，直至满足停止条件。

(4) 将输入空间划分为 M 个区域 $R_1, R_2, \dots, R_M$ ，生成决策树：

$$f(x) = \sum_{m=1}^M \hat{c}_m I(x \in R_m)$$



切分点 s 通常取两个数据值的均值

5.3.2 分类树的生成

对分类树用基尼指数 (Gini index) 最小化准则

基尼指数

分类问题中，假设有 K 个类，样本点属于第 k 类的概率为 p_k ，则概率分布的基尼指数定义为：

$$\text{Gini}(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

从实际计算的角度，对于给定的样本集合 D ，其基尼指数为：

$$\text{Gini}(D) = 1 - \sum_{k=1}^K \left(\frac{|C_k|}{|D|} \right)^2$$

这里， C_k 是 D 中属于第 k 类的样本子集， K 是类的个数。

如果样本集合 D 根据特征 A 是否取某一可能值 a 被分割成 D_1 和 D_2 两部分，则在特征 A 的条件下，集合 D 的基尼指数定义为：

$$\text{Gini}(D, A) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2)$$

分类树生成算法：根据训练数据集，从根结点开始，递归地对每个结点进行以下操作，构建二叉决策树：

(1) 设结点的训练数据集为 D ，计算现有特征对该数据集的基尼指数。此时，对每一个特征 A ，对其可能取的每个值 a ，根据样本点对 $A=a$ 的测试为“是”或“否”将 D 分割成 D_1 和 D_2 两部分，计算 $A=a$ 时的基尼指数。

(2) 在所有可能的特征 A 以及它们所有可能的切分点 a 中，选择基尼指数最小的特征及其对应的切分点作为最优特征与最优切分点。依最优特征与最优切分点，从现结点生成两个子结点，将训练数据集依特征分配到两个子结点中去。

(3) 对两个子结点递归地调用 (1)，(2)，直至满足停止条件。

(4) 生成 CART 决策树。

算法停止计算的条件是结点中的样本个数小于预定阈值，或样本集的基尼指数小于预定阈值（样本基本属于同一类），或者没有更多特征。



Tips：算法停止计算的约束条件，其实也是在变相的剪枝。

剪枝分为预剪枝（Pre-pruning）和后剪枝（Post-pruning），常见的预剪枝有强制限制树的深度、内部节点再划分所需最小样本数、叶子节点最少样本数、最大叶子节点数等；后剪枝见下文的代价复杂度剪枝（CCP）算法。

5.3.3 CART 剪枝

算法流程：

(1) 设 $k=0$ ， $T = T_0$ 。

(2) 设 $\alpha = +\infty$ 。

(3) 自下而上地对各内部所有结点 t 计算 $C(T_t)$ ， $|T_t|$ 以及

$$g(t) = \frac{C(t) - C(T_t)}{|T_t| - 1}$$
$$\alpha = \min(\alpha, g(t))$$

T_t 表示以 t 为根结点的子树， $C(T_t)$ 是对训练数据的预测误差， $|T_t|$ 是 T_t 的叶结点个数。

$C(t) - C(T_t)$ 表示对于内部节点 t 来说，新树（只有 t 一个节点）与旧树（以 t 为根节点）在训练集上面的误差的改变量。

(4) 自上而下地访问内部结点 t ，如果有 $g(t) = \alpha$ ，进行剪枝，并对叶结点 t 以多数表决法决定其类，得到树 T 。

一旦 $g(t) = \alpha$ ，就表示新树（只有 t 一个节点）与旧树（以 t 为根节点）有相同的损失，此时就可以剪枝，选取节点更少的 t 。

(5) 设 $k=k+1$ ， $\alpha_k = \alpha$ ， $T_k = T$ 。

(6) 如果 T 不是由根结点单独构成的树，则回到步骤 (4)。

(7) 采用交叉验证法在子树序列 $T_0, T_1, \dots, T_n$ 中选取最优子树 T_α 。

说人话：

1. 假设最开始内部节点有 N 个，减去根节点有 $N-1$ 个。
2. 第一次会遍历 $N-1$ 个内部节点，算 g 值，得到 $N-1$ 个 g 值，最终会至少剪 1 个枝（因为最小值一定会有一个），剩 $N-2$ 个内部节点；
3. 随后进行第二次递归，剩最少 $N-3$ 个节点；
4. 依次往复，直到只剩下根节点。
5. 此时会产生 $N-1$ 颗子树，对每一棵树，通过 K 折交叉验证来算训练集上的损失，取最小的那个损失对应的子树就是最终的输出。

5.3.4 预剪枝与后剪枝的对比

维度	CCP 后剪枝	预剪枝（max_depth 等）
原理	先长满再剪	限制生长条件
计算成本	高（训练多棵树）	低（训练一棵树）
效果	理论上更优	通常足够好
调参	自动选择 alpha	手动调 max_depth 等
推荐场景	追求极致性能	日常开发

5.4 代码

<https://github.com/Unagi-cq/th-nlp-learning/tree/main/1.%20%E7%BB%9F%E8%AE%A1%E5%AD%A6%E4%B9%A0%E6%96%B9%E6%B3%95/codes>

5.5 思考

问题：给定任意数量级的样本，决策树都能做到构建出一颗完全覆盖住样本集的树吗？

答：不能。样本可能出现有冲突的情况，比如 2 条样本具有同样的特征，但是有两个不同的标签。

6. 第六章 逻辑斯蒂回归与最大熵模型

逻辑斯蒂回归（logistic regression，LR）是传统机器学习领域**非常经典**的模型，LR 是判别式模型，虽然叫做回归但是它是分类模型。

LR 凭借其白盒化的权重机制、超快的推理速度，曾经统治着早期时代的搜广推领域，后来被 DNN 所超越替代。但是由于其超强的可解释性，在如今的搜推业务中仍然存在。

 LR不仅是搜广推领域的基石，它也是如今大部分NLP模型（最后一层分类网络）的基础。

LR 其实就是在线性回归的基础上，套一个 sigmoid 函数或者 softmax 函数。即使在以后做深度学习任务或者是大模型，其实都是有 LR 这一步在的。比如，再大的 LLM，无论是 Decoder Only 还是 Encoder Only，它最后都是要预测词表的，这一步的结构其实就是一个**线性层（线性回归+softmax）**。

而最大熵模型属于信息学范畴，它与 LR 的关系如下：

- 二分类的最大熵模型 等价于 逻辑回归（Logistic Regression）。
- 多分类的最大熵模型 等价于 多项逻辑回归（Softmax Regression）。

 在原书，作者更想表达的是 LR 的一般形式，极力地把 softmax 和 sigmoid 表达为通用形式，所以比较难懂。建议大家抛弃原书本章节，直接从二分类、多分类的角度来学习 LR，事半功倍。在日后的深度学习、网络博客中也极少见到书中的表述，一般都是直接从损失说起。

6.1 逻辑回归 LR

让我们直接忽略原书的逻辑斯蒂分布，从模型说起。

- 二分类：只有 2 个类别，正或负，一个样本只能是正或负。
- 多分类：类别集合有多个类别，一个样本只取其中的一个类别。
- 多标签分类：类别集合有多个类别，一个样本可能有一或者多个类别。

6.1.1 二项逻辑回归

二项逻辑斯蒂回归，其实就是二分类模型，它的表现形式如下：

$$P(y = 1|x) = \sigma(wx + b) = \text{sigmoid}(wx + b) = \frac{1}{1 + e^{-(wx+b)}}$$

- 也就是说，给定输入 x ， $w x + b$ 外面套一个 sigmoid 等于 p ，这个值 p 我们约定是正例（也就是 $y=1$ ）的概率，那么负例的概率就是 $1-p$ 。

上述二分类模型的表述，有非常多值得推敲的点，想要掌握二项逻辑回归，必须流畅回答以下问题。

问题 1：为什么二分类不用 softmax 或者是其他非线性函数如 tanh、relu，而用 sigmoid？

回答：给定 x 后， $w x + b$ 并不是一个概率值，我们期望把它映射到 01 区间内，因此，需要一个函数来做归一化。从统计学上考虑，一个事件是正负的概率之和肯定是 1，因此这个归一化函数要能支持 $F(x) + F(-x) = 1$ 。最后由于需要求梯度来优化参数，这个函数还要能够可导。

tanh、relu 不满足上述性质。除此之外，当 $n=2$ 时，softmax 其实就是 sigmoid。

问题 2：为什么得到正例的概率 p 后，就可以说负例是 $1-p$ ，严谨吗？

回答：带入 sigmoid 公式，可得 $F(x) + F(-x) = 1$ 。

问题 3：如果大家对多分类（类别>2）有所了解，一定知道多分类最终输出的是每个类别的概率。那么，二分类为什么不分别输出正类的概率和负类的概率？或者说为什么 $\text{sigmoid}(wx + b)$ 的值不可以是负类的概率呢？

回答：其实是可以的，但是没有必要。首先，采用了 sigmoid 后，另外一个概率天然就是 $1 - p$ 的关系，这样我们构建的模型可以减少不必要计算，提高效率。再者，如果非让 $\text{sigmoid}(wx + b)$ 的值是负类的概率也可以，但是在所有的论文、资料中，大家都是默认它是正类也就是 $y=1$ 的概率。

二分类的损失函数。（这一步必须要能够从交叉熵函数一步一步推导出来，需倒背如流。）

已知交叉熵公式为：

$$H(P, Q) = - \sum P(x) \log Q(x)$$

设样本 x 的真实标签为 y ，预测输出为 $\hat{y}$ ，则真实分布 $P(x)$ 为：

$$P(y = 1|x) = y; P(y = 0|x) = 1 - y$$

预测分布 $Q(x)$ 为：

$$Q(y = 1|x) = \hat{y}; Q(y = 0|x) = 1 - \hat{y}$$

代入交叉熵函数可得损失函数为：

$$L = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

求梯度时，sigmoid 会起到很巧妙的作用（会产生消项）。L 对 w 求偏导：

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right) \cdot \hat{y}(1-y) \cdot x = x(\hat{y} - y)$$

6.1.2 多项逻辑回归

多项逻辑斯蒂回归，就是多分类模型，它的表现形式如下：

$$P(y = k|x) = \text{softmax}(W_k x + b_k) = \frac{e^{w_k x + b_k}}{\sum_{j=1}^K e^{w_j x + b_j}}$$

其中 K 是类别总数， w_k 和 b_k 是第 k 类的权重和偏置。

多分类损失函数。（需倒背如流。）

已知交叉熵公式为：

$$H(P, Q) = -\sum P(x) \log Q(x)$$

设样本 x 的真实标签为 y ，预测输出为 y_i ，则真实分布 $P(x)$ 表达为样本 x 是不是类别 k ：

$$P(y = k|x) = y_k$$

在实际代码中， **$P(x)$ 一般用独热向量来表示。**

预测分布 $Q(x)$ 为：

$$Q(y = k|x) = \hat{y}_k$$

因此，多分类的损失为：

$$L = -y_k \log(\hat{y}_k)$$

在上式中，因为只有 y_k 为1，其他都为0，所以就只保留了 y_k 这一项。



二分类和多分类的损失函数非常重要。在torch库实现时，二分类用的是 `BCEWithLogitsLoss`，真实标签传入的是一个标量，多分类用的是 `CrossEntropyLoss`，真实标签传入的是标签的独热向量。

6.2 最大熵模型（Maximum Entropy Model）

仅作了解，知道最大熵模型与逻辑回归的等价关系即可。

最大熵模型的原理是：在满足已知约束条件下，选择熵最大的概率分布。

最大熵建立在以下的假设之上：

1. **最保守的估计**：不做任何未经证实的假设
2. **均匀分布熵最大**：在没有任何信息时，假设所有可能等概率
3. **满足约束的前提下最不确定**：避免引入额外偏见

最大熵模型形式

给定特征函数 $f_k(x, y)$ 和期望约束：

$$E_P[f_k] = E_{\tilde{P}}[f_k] = \sum_{x,y} \tilde{P}(x, y) f_k(x, y)$$

最大熵模型的解为：

$$P(y|x) = \frac{1}{Z(x)} \exp \left(\sum_k \lambda_k f_k(x, y) \right)$$

其中 $Z(x) = \sum_y \exp \left(\sum_k \lambda_k f_k(x, y) \right)$ 是归一化因子。

6.3 模型优化算法

模型的优化算法是一门专有研究学科，从机器学习早期的迭代尺度法、牛顿法、拟牛顿法、L-BFGS，再到深度学习早期的随机梯度下降法SGD、Mini-batch GD，再到目前训练大模型所用的Adam、AdamW。

模型梯度优化算法可以去了解SGD和Adam。迭代尺度法、牛顿法可以不看。

做深度学习先用 Adam，如果效果遇到瓶颈且是视觉任务，尝试切换 SGD；做传统小模型直接用 L-BFGS。

6.4 代码实现

<https://github.com/Unagi-cq/th-nlp-learning/tree/main/1.%20%E7%BB%9F%E8%AE%A1%E5%AD%A6%E4%B9%A0%E6%96%B9%E6%B3%95/codes>

本笔记pdf由【三黄工作室】微信公众号提供。欢迎关注。



7. 第七章 支持向量机

支持向量机（SVM）是一个较为经典的机器学习模型，它是二类分类模型，判别式。

在日常工作中，我还并没使用过或者见到过别人使用过SVM来解决实际问题。并且SVM相较于LR更“黑盒”，SVM对高维稀疏的场景下表现不佳，其输出的是+1/-1，不像LR输出的是概率。

因此本章**主要了解**SVM的原理、硬/软间隔、线性SVM、非线性SVM、对偶问题即可，满足基本的面试和知识面所需。

7.1 线性可分SVM与硬间隔最大化

学习SVM，可以从第二章的感知机理论开始类比。

感知机是最早的线性分类器，其核心是寻找一个超平面，将两类数据完全分开。假设我们有训练样本 (x_i, y_i) ，其中 $y_i \in \{-1, +1\}$ ，感知机模型的决策函数是：

$$f(x) = \text{sign}(w \cdot x + b)$$

训练目标是找到一组 w, b ，使得所有样本被正确分类。感知机的分类平面是无穷的，只要能把正负样本分割开， w 和 b 可以取满足条件的任何值。

线性可分SVM就是在感知机的基础上，通过加一定的约束条件（最大化几何间隔），来使这样的平面唯一。

7.1.1 函数间隔

对一个超平面 $w \cdot x + b = 0$ 。

样本点 (x_i, y_i) 到超平面的**函数间隔**为：

$$\hat{\gamma}_i = y_i(w \cdot x_i + b)$$

整个数据集到超平面的**函数间隔**是离超平面最近的样本点到超平面的距离：

$$\hat{\gamma} = \min_i \hat{\gamma}_i$$

感知机在优化的时候，算的其实就是点到超平面的函数间隔（对函数间隔的大小没有要求，只要求正负符号）。由于参数 w 、 b 同时缩放常数倍，超平面不变，所以感知机的超平面是无限的。

7.1.2 几何间隔

SVM 的目标不仅是要能分开正负样本，而且还要**选择间隔最大的超平面**。

由于参数 w 、 b 同时缩放常数倍，超平面不变，但是函数间隔会变，所以提出对参数 w 进行归一化，这样函数间隔就能够得到一个唯一值，称之为**几何间隔**。

归一化的方法就是令参数 w 的模为1：

$$\|w\| = 1$$

此时，样本点 (x_i, y_i) 到超平面的**几何间隔**为：

$$\gamma_i = y_i \left(\frac{w}{\|w\|} \cdot x_i + \frac{b}{\|w\|} \right)$$

整个数据集到超平面的**几何间隔**是离超平面最近的样本点到超平面的距离：

$$\gamma = \min_i \gamma_i$$

7.1.3 线性可分SVM的算法描述

考虑如何求得一个几何间隔最大的分离超平面，即最大间隔分离超平面。这个问题可以表示为下面的约束最优化问题：

$$\begin{aligned} & \max_{w,b} \gamma \\ \text{s.t.} \quad & y_i \left(\frac{w}{\|w\|} \cdot x_i + \frac{b}{\|w\|} \right) \geq \gamma, \quad i = 1, 2, \dots, N \end{aligned}$$

由于几何间隔和函数间隔存在换算关系，并且函数间隔可以约定为1。最终求解线性可分SVM就变成了求解约束最优化问题：

$$\begin{aligned} & \min_{w,b} \frac{1}{2} \|w\|^2 \\ \text{s.t.} \quad & y_i (w \cdot x_i + b) - 1 \geq 0, \quad i = 1, 2, \dots, N \end{aligned}$$

上面的约束条件是一个带不等式约束的凸二次规划（Quadratic Programming, QP）问题。

在线性可分情况下，训练数据集的样本点中与分离超平面距离最近的样本点的实例称为**支持向量**。支持向量是使约束条件式等号成立的点，即

$$y_i (w \cdot x_i + b) - 1 = 0$$

对 $y_i = +1$ 的正例点，支持向量在超平面

$$H_1 : w \cdot x + b = 1$$

上，对 $y_i = -1$ 的负例点，支持向量在超平面

$$H_2 : w \cdot x + b = -1$$

上。 H_1 与 H_2 之间的距离称为间隔。间隔依赖于分离超平面的法向量 w ，等于 $\frac{2}{\|w\|}$ 。 H_1 和 H_2 称为间隔边界。



假设输入 x 只有2维，理想情况下只需要2个点就能把超平面给固定下来了。

7.1.4 线性可分SVM的对偶算法

对于刚才提到的QP问题，虽然有一些python库可以直接或近似求解，比如 `cvxpy`、`scipy`，但**对偶形式具有巨大优势**：

1. 对偶形式可以引入核函数
2. 对偶形式有高效的求解算法SMO，原始形式在数据量大时难以计算

线性可分SVM的对偶最优化问题如下：

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0 \\ & \alpha_i \geq 0, \quad i = 1, 2, \dots, N \end{aligned}$$

则可按下式求得原始最优化问题的解 w^*, b^* ：

$$\begin{aligned} w^* &= \sum_{i=1}^N \alpha_i^* y_i x_i \\ b^* &= y_j - \sum_{i=1}^N \alpha_i^* y_i (x_i \cdot x_j) \end{aligned}$$

7.2 线性不可分SVM与软间隔最大化

之前我们讨论的**硬间隔**要求所有样本必须被正确分类且位于间隔边界之外。但这在现实中几乎是不可能的，因为：

1. **噪声存在**：数据中难免有标注错误或异常点。
2. **线性不可分**：大多数真实数据并不是严格线性可分的。

如果强行使用硬间隔 SVM，为了容纳一个异常点，超平面可能会发生剧烈偏转，导致模型**过拟合**，泛化能力极差。

软间隔 SVM 的核心思想是：允许少量样本不满足约束条件（即允许误分类或落入间隔内），但要对这些“违规”行为进行惩罚。

为了量化每个样本“违规”的程度，为每个样本 (x_i, y_i) 引入一个非负的松弛变量 $\xi_i \geq 0$ 。

原来的硬间隔约束：

$$y_i(w \cdot x_i + b) \geq 1$$

现在的软间隔约束变为：

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

ξ_i 的几何意义：

- $\xi_i = 0$ ：样本点满足硬间隔约束，位于间隔边界外侧且分类正确。
- $0 < \xi_i \leq 1$ ：样本点位于间隔边界内部，但在分离超平面的正确一侧（被“容忍”了，但没有误分类）。
- $\xi_i > 1$ ：样本点被分错了（位于分离超平面的错误一侧）。

7.2.1 线性不可分SVM的算法描述

既然允许犯错，就不能无限制地犯错。我们需要在“间隔最大化”和“错分程度最小化”之间寻找平衡。通过引入超参数 C 来调节这两者的权重，得到最终的原始优化问题：

$$\begin{aligned} \min_{w, b, \xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad i = 1, 2, \dots, N \\ & \xi_i \geq 0, \quad i = 1, 2, \dots, N \end{aligned}$$

7.2.2 线性不可分SVM的对偶算法

利用拉格朗日乘子法，最终得到的对偶问题与硬间隔惊人地相似，唯一的区别在于 α_i 的上界：

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0 \\ & 0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, N \end{aligned}$$

那么可按式求得原始最优化问题的解 w^*, b^* ：

$$w^* = \sum_{i=1}^N \alpha_i^* y_i x_i$$

$$b^* = y_j - \sum_{i=1}^N y_i \alpha_i^* (x_i \cdot x_j)$$

🐼 对于线性不可分SVM的支持向量，可能出现在任何位置，无论是函数间隔内、分割平面上或是其他地方。

7.2.3 线性不可分SVM的合页损失视角

软间隔最大化的原始目标函数其实等价于最小化 **Hinge Loss（合页损失）** 加上 L2 正则化：

$$\min_{w,b} \sum_{i=1}^N \max(0, 1 - y_i(w \cdot x_i + b)) + \lambda \|w\|^2$$

`sklearn.svm.LinearSVC` 中有一个 `loss='hinge'` 的参数选项，它本质上就是在解软间隔 SVM 的原始问题。

7.3 非线性SVM与核函数

7.3.1 核技巧

在前面的讨论中，我们假设数据是**线性可分**的（或者近似线性可分，通过软间隔处理）。但在现实世界中，很多数据（如异或问题 XOR、同心圆分布等）在原始输入空间中是**线性不可分**的。无论你怎么画直线（或超平面），都无法完美分开它们。

如果数据在低维空间分不开，那就把它映射到高维空间。在高维空间中，数据往往会变得线性可分。

将原始输入空间 $\mathcal{X}$ 中的样本 x ，通过一个非线性映射 $\phi(x)$ ，映射到一个更高维（甚至是无穷维）的特征空间 $\mathcal{H}$ ：

$$x \in \mathbb{R}^n \xrightarrow{\phi} \phi(x) \in \mathcal{H}$$

在这个高维空间 $\mathcal{H}$ 中，构建一个**线性**分离超平面：

$$w \cdot \phi(x) + b = 0$$

也就是说，设计一个函数，对输入 x 先做变换，然后再按照正常的流程计算。

但直接这样做有两个比较严重的问题：

1. **维数太高**：如果映射到很高维（例如多项式核映射到 d 次方，维度可能是 n^d ），甚至无穷维（如 RBF 核），根本无法存储向量 $\phi(x)$ 和权重 w 。
2. **计算太慢**：在对偶问题的目标函数和决策函数中，都需要计算内积 $\phi(x_i) \cdot \phi(x_j)$ 。如果先算出高维向量再做点积，计算量是天文数字。

因此，引出核函数。**核技巧**是 SVM 最精妙的设计。

在 SVM 的对偶形式中，训练和预测**只依赖于样本之间的内积** $\phi(x_i) \cdot \phi(x_j)$ ，而不需要显式地知道 $\phi(x)$ 是什么，也不需要算出高维向量本身。

如果我们能找到一个函数 $K(x, z)$ ，使得它在低维空间的计算结果 **等于** 高维空间的内积：

$$K(x, z) = \phi(x) \cdot \phi(z)$$

那么，我们就可以**直接在低维空间计算** $K(x, z)$ ，从而隐式地在高维空间进行操作！这就是**核函数**。也就是说，不需要知道 $\phi(x)$ 到底是什么函数公式的，反正我们在对偶形式里算的是内积，直接假定 $\phi(x) \cdot \phi(z)$ 是什么函数公式就可以了。

引入核函数后，SVM 的对偶优化问题变为：

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(x_i, x_j) \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C \end{aligned}$$

最终的决策函数变为：

$$f(x) = \text{sign} \left(\sum_{i=1}^N \alpha_i^* y_i K(x_i, x) + b^* \right)$$

整个过程中，我们从未真正进入过高维空间 $\mathcal{H}$ ，也从未见过 $\phi(x)$ ，但我们却享受了高维空间线性可分的好处。

7.3.2 常用的核函数

由于sklearn库使用的比较多，直接介绍 `sklearn.svm.SVC` 中内置的几种常用核函数。

7.3.2.1 线性核

$$K(x, z) = x \cdot z$$

不做映射，就在原始空间做线性分类。

7.3.2.2 多项式核

$$K(x, z) = (\gamma x \cdot z + r)^d$$

将数据映射到 d 次多项式特征空间。参数较多，调参较难，高次多项式容易导致数值不稳定。

7.3.2.3 高斯径向基核

$$K(x, z) = \exp(-\gamma \|x - z\|^2)$$

将数据映射到**无穷维**空间。性能通常最好，是默认首选。

7.3.2.4 Sigmoid 核

$$K(x, z) = \tanh(\gamma x \cdot z + r)$$

源自神经网络，相当于两层感知机。在某些参数下不是正定核，现在用得较少。

7.3.3 正定核

正定核是一个保证了“核技巧”在数学上合法、且优化问题有唯一全局最优解的函数。

如果一个函数 $K(x, z)$ 不是正定核，那么它可能无法对应任何真实的高维空间映射，或者会导致 SVM 的优化问题变成非凸问题（陷入局部最优，无法求解）。

在线性代数中，一个对称矩阵 A 被称为**正定矩阵**，如果对于任意非零向量 c ，都有：

$$c^T A c > 0$$

几何意义上，这意味着该矩阵对应的二次型曲面是一个开口向上的“碗”，拥有唯一的最低点（全局最小值）。

常见的基本正定核

以下函数已被证明是正定核：

- **线性核**： $K(x, z) = x^T z$
- **多项式核**： $K(x, z) = (\gamma x^T z + r)^d$ ($\gamma > 0, r \geq 0, d \in \mathbb{N}$)
- **高斯 RBF 核**： $K(x, z) = \exp(-\gamma \|x - z\|^2)$ ($\gamma > 0$)
- **拉普拉斯核**： $K(x, z) = \exp(-\gamma \|x - z\|_1)$

7.4 代码实现

<https://github.com/Unagi-cq/th-nlp-learning/tree/main/1.%20%E7%BB%9F%E8%AE%A1%E5%AD%A6%E4%B9%A0%E6%96%B9%E6%B3%95/codes>

7.5 思考

问题：sklearn库是怎么求解svm的？

答：sklearn)本身并不从头编写 SVM 的求解算法，而是作为封装器 (Wrapper)，调用了两个非常著名的、用 C/C++ 编写的高性能底层库：

1. `libsvm`：用于 `SVC`，`NuSVC`，`SVR`，`NuSVR`，`OneClassSVM`。
2. `liblinear`：用于 `LinearSVC`，`LinearSVR`。

sklearn库有SVR和SVC，一个是用来回归一个是分类，我们只说分类。分类的模型中，有两个重要的包：

1. `sklearn.svm.SVC`
2. `sklearn.svm.LinearSVC`

`sklearn.svm.SVC` 求解的是对偶形式，核心算法是SMO，基于libsvm优化，可以设置核函数。

`sklearn.svm.LinearSVC` 求解的是原始形式，核心算法是坐标下降法或截断牛顿法，基于liblinear优化，默认使用平方合页损失。

问题：为什么sklearn库里面的 `sklearn.svm.LinearSVC` 不支持核函数而 `sklearn.svm.SVC` 支持？

答：因为 `sklearn.svm.LinearSVC` 求解的是原始形式，只有对偶形式才能使用核函数。

8. 第八章 提升方法

提升方法是集成学习中的一个分支。

集成学习是指通过构建并结合多个学习器（弱学习器）来完成学习任务，从而获得比单一学习器显著优越的泛化性能。集成学习主要分为两大类：

- Bagging：随机森林 2001
- Boosting
 - 基于样本调整权重：AdaBoosting 1997
 - 基于残差拟合：GBDT 2001、XGBoost 2016、LightGBM 2017

这些模型中，**XGBoost非常重要**，它的效果很好、很快，也具有解释性，某些情况下是DNN的替代方案，在如今的生产实践中仍然有一席之地，务必要掌握。

 由于著书时间的原因，本章作者只提到了AdaBoosting和提升树，没有介绍后来新兴的XGB模型。

8.1 AdaBoosting

Boosting本质上是一种“元算法”或“框架思想”，它定义了如何组合多个模型、如何迭代更新权重（或拟合残差），但并不限制底层基学习器的具体类型。

只要基学习器满足一个核心条件：比随机猜测略好（即准确率 > 50%，称为“弱学习器”），理论上都可以作为 Boosting 的基组件。

8.1.1 基本思想

对于底层基学习器，理论上我们可以任意替换，如LR、KNN、决策树等。但是对于若干个基学习器每迭代一轮后，如何做优化，是Boosting需要考虑的问题。具体来说，**AdaBoosting明确了下面两个问题**：

1. 在每一轮后，提高那些被前一轮弱分类器错误分类的样本权重，降低那些被前一轮弱分类器成功分类的样本权重
2. 对于如何组合多个弱分类器，AdaBoosting采用加权多数表决的方法，加大分类误差率小的弱分类器的权值，使其在表决中起较大的作用，减小分类误差率大的弱分类器的权值，使其在表决中起较小的作用。

AdaBoosting是早期**基于样本调整权重**的方法。

8.1.2 算法描述

(1) 初始化训练数据的权值分布

$$D_1 = (w_{11}, \dots, w_{1i}, \dots, w_{1N}), \quad w_{1i} = \frac{1}{N}, \quad i = 1, 2, \dots, N$$

(2) 对 $m = 1, 2, \dots, M$

(a) 使用具有权值分布 D_m 的训练数据集学习，得到基本分类器

$$G_m(x) : \mathcal{X} \rightarrow \{-1, +1\}$$

(b) 计算 $G_m(x)$ 在训练数据集上的分类误差率

$$e_m = P(G_m(x_i) \neq y_i) = \sum_{i=1}^N w_{mi} I(G_m(x_i) \neq y_i)$$

(c) 计算 $G_m(x)$ 的系数

$$\alpha_m = \frac{1}{2} \log \frac{1 - e_m}{e_m}$$

(d) 更新训练数据集的权值分布

$$D_{m+1} = (w_{m+1,1}, \dots, w_{m+1,i}, \dots, w_{m+1,N})$$

$$w_{m+1,i} = \frac{w_{mi}}{Z_m} \exp(-\alpha_m y_i G_m(x_i)), \quad i = 1, 2, \dots, N$$

$$Z_m = \sum_{i=1}^N w_{mi} \exp(-\alpha_m y_i G_m(x_i))$$

(3) 构建基本分类器的线性组合

$$f(x) = \sum_{m=1}^M \alpha_m G_m(x)$$

得到最终分类器

$$G(x) = \text{sign}(f(x)) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$$

以上是AdaBoosting方法的数学描述。

下面我们通过代码来理解。

代码块

```
1  def fit(self, X, y):
2      """
3      训练AdaBoost模型
4      :param X: (n_samples, n_features) 的输入数据
5      :param y: (n_samples,) 的标签, 取值为 +1 或 -1
6      """
7      n_samples = X.shape[0]
8
9      # 初始化样本权重, 均匀分布
10     weights = np.ones(n_samples) / n_samples
11
12     for i in range(self.n_estimators):
13         # 训练弱学习器
14         model = self.trainer.fit(X, y, sample_weight=weights)
15
16         # 预测
17         preds = model.predict(X)
18
19         # 计算加权错误率
20         miss = (preds != y).astype(int)
21         error = np.dot(weights, miss) / np.sum(weights)
22
```

```

23         # 计算弱学习器权重
24         #  $\alpha = 1/2 * \ln((1 - error) / error)$ 
25         if error == 0:
26             alpha = 10.0 # 防止除零
27         else:
28             alpha = self.learning_rate * 0.5 * np.log((1 - error) / error)
29
30         # 更新样本权重
31         #  $weights = weights * \exp(-\alpha * y * preds)$ 
32         weights = weights * np.exp(-alpha * y * preds)
33         weights = weights / np.sum(weights) # 归一化
34
35         self.models.append(model)
36         self.model_weights.append(alpha)
37
38         # 如果错误率接近0, 提前结束
39         if error < 1e-10:
40             break

```

至此，AdaBoost 的执行流程就闭环了：

初始化权重 → **训练弱分类器** → **算误差** → **算分类器权重 α** → **更新样本权重 w** → **循环** → **线性组合输出**。

 **AdaBoosting执行时弱分类器是串行优化、保存的。**例如，初始化一个树模型，在第一次优化后，保存第一次优化树模型到数组中；下一次再拿前一次的结果做优化，保存第二次优化树模型到数组中；循环直到优化次数耗尽或者损失为0。

最终预测时，第1~n次优化的模型都会参与加权预测。

这就是前向分布算法思想。

8.2 提升树

提升树（Boosting Tree） 是 Boosting 思想与决策树（Decision Tree）模型结合的产物。

简单来说，**提升树 = 加法模型 + 前向分步算法 + 决策树**。

提升树（尤其是XGB）是目前工业界（尤其是处理结构化数据时）最强大、应用最广泛的机器学习方法之一。GBDT、XGBoost、LightGBM，本质上都是提升树的不同进化版本。

8.2.1 算法定义

提升树利用**加法模型**与前向分步算法，将多个**决策树**（通常是 CART 回归树）组合起来。

- **基学习器**：决策树（CART）。
- **模型形式**：
$$f_M(x) = \sum_{m=1}^M T(x; \Theta_m)$$
 - $T(x; \Theta_m)$ 表示第 m 棵树。
 - Θ_m 是树的参数（分裂点、叶子节点值等）。

8.2.2 提升树与AdaBoosting的对比

特性	AdaBoost	GBDT (Gradient Boosting)
迭代依据	样本权重 (Sample Weights)	残差 / 负梯度 (Residuals / Negative Gradient)
如何关注错误	提高被分错样本的权重，降低分对样本的权重。下一轮训练时，模型会“被迫”更关注高权重的难样本。	计算当前模型预测值与真实值的差（残差），下一棵树直接把这个差值作为目标值（Label）去拟合。
基学习器	理论上可以是任意分类器，但常用决策树桩（深度为 1 的树）。	必须是CART 回归树（即使做分类任务，内部也是用回归树拟合残差）。
最终组合方式	加权投票（分类）或加权求和。每个树有一个系数，误差越小的树权重越大。	线性累加。所有树的预测结果直接相加（），通常每棵树的权重由学习率控制，树本身没有单独的。
损失函数	固定为指数损失函数 (Exponential Loss)。	任意可微损失函数（如平方损失、绝对值损失、Huber 损失、对数损失等）。这是 GBDT 最大的优势。

我们都知道，决策树CART的构建过程中是不存在梯度的，对于给定的数据集，生成的决策树也是固定的。

那么GBDT是如何利用梯度来优化CART的？

GBDT 并没有改变单棵 CART 树的内部生成算法，它改变的是这棵树要拟合的“目标值（Label）”。也就是说，梯度不是用来指导树怎么“分裂”的，而是用来决定树要“预测什么”的。

因为GBDT是加法模型，如果第1次优化得到 m_1 模型， m_1 模型的损失的梯度会作为 m_2 模型的预测值，然后得到 m_2 模型，接下来 $m_1 + m_2$ 模型的损失的梯度会作为 m_3 模型的预测值，然后得到 m_3

模型，依次循环...

最后来看一下提升树家族。

阶段	代表算法	核心特点	优缺点
第一代	AdaBoost + 树	基于样本权重调整。每棵树关注上一棵树分错的样本。	对异常值敏感；只能处理特定损失函数。
第二代	GBDT (Friedman, 2001)	基于负梯度拟合。将 Boosting 推广到任意可微损失函数。	通用性强；但训练速度较慢，缺乏正则化。
第三代	XGBoost (陈天奇, 2016)	二阶泰勒展开 + 正则化。在 GBDT 基础上引入二阶导数信息，并显式加入正则项防止过拟合。	精度极高；支持并行化（建树过程）；工程实现极其优秀。
第四代	LightGBM (微软, 2017)	直方图算法 + 采样优化。针对大数据量，通过近似分裂点和梯度采样大幅提速。	速度极快；内存占用低；适合海量数据。
第五代	CatBoost (Yandex, 2018)	类别特征处理 + 对称树。专门优化了类别特征的处理，解决了梯度偏差问题。	调参简单；对类别特征友好；预测速度快。

XGB以及后续模型暂时不再介绍。

8.3 代码实现

<https://github.com/Unagi-cq/th-nlp-learning/tree/main/1.%20%E7%BB%9F%E8%AE%A1%E5%AD%A6%E4%B9%A0%E6%96%B9%E6%B3%95/codes>

8.4 思考

问：为什么提升方法的弱学习器一般是决策树，而很少用LR、KNN、SVM等呢？

答：提升方法之所以偏爱决策树（CART），本质原因是：它最适合做“逐步修正误差”的函数拟合器。提升方法要求弱学习器能做回归、能拟合任意形状的函数。其本质是想要

决策树具有强大的非线性拟合能力，是分段函数，搭配上提升方法可以理解为每棵树负责修一小块“错误区域”。第1棵树：大致拟合趋势；第2棵树：修正一块错误区域；第3棵树：再修一块。**对局部误差进行精细修正。**

- LR本身学习的是线性关系，累加后仍然是线性。
- SVM训练成本高，本身就是强学习器，且不适合逐步拟合梯度。
- KNN没有训练过程，无法拟合梯度。

问：提升方法只要弱学习器效果好不就行了吗？如何调参？

答：在提升方法中，弱学习器必须“适度弱”，整体才会最强。Boosting 的机制是**逐步叠加修正**，如果每个模型都很强：第一棵树就把数据拟合得很好（甚至过拟合），后面的树学不到“剩余结构”、只会拟合噪声，会导致**过拟合 + 泛化差**。

9. 第十章 隐马尔可夫模型

原书第九章 EM算法、第十章 条件随机场实际应用比较少，这里就直接跳过了。隐马尔可夫模型也仅作了解。

隐马尔可夫模型（Hidden Markov Model, HMM）是一种统计模型，是标准的**生成式模型**，广泛应用于**序列数据建模**。HMM 在如词性标注、命名实体识别、语音识别等中有着经典的应用。

9.1.1 算法概念

HMM 描述了一个由**隐含状态序列**生成**观测序列**的过程。它基于两个基本假设：

- **齐次马尔可夫假设**：当前时刻的隐藏状态只依赖于前一时刻的隐藏状态（一阶马尔可夫链）。
- **观测独立性假设**：当前时刻的观测值只依赖于当前时刻的隐藏状态。

模型由五元组 $\lambda = (S, V, \pi, A, B)$ 定义：

- S ：隐藏状态集合（例如：词性标签 {名词, 动词...}）。
- V ：观测值集合（例如：单词本身）。
- π ：初始状态概率分布。
- A ：状态转移概率矩阵 ($P(s_t | s_{t-1})$)。
- B ：观测概率矩阵（发射概率， $P(o_t | s_t)$ ）。

9.1.2 HMM 的三大基本问题

这三个问题对应了不同的算法：

问题类型	描述	核心算法	应用场景示例
概率计算	给定模型 λ 和观测序列 O ，计算 $P(O \lambda)$ 。	前向 - 后向算法	评估某个句子由该模型生成的可能性。
学习问题	给定观测序列 O ，估计模型参数 λ 以最大化 $P(O \lambda)$ 。	Baum-Welch 算法 (EM 算法的特例)	在无标注数据上训练模型参数。
预测问题	给定模型 λ 和观测序列 O ，求最可能的隐藏状态序列 I 。	维特比算法 (Viterbi, 动态规划)	词性标注：已知句子，求最可能的词性序列。

文档介绍

本笔记pdf由【三黄工作室】微信公众号提供。欢迎关注。

